

FOUNDATION & UTILS — Requirements, Design & Tasks v3.0

Source: FOUNDATION_UTILS_UPDATE_v2_0.md + all v3.0 feature specs **Scope:** shared/crate — Type Registry, Helper Registry, DeterministicContext, Compiler Config, Cross-System Traits **Consumers:** Praman (123), Tejas (82), Dhruva (50), Siddhanta (38), Dristi (122) — 415 total features **Pipeline:** REQ → DES → TASK (every requirement traceable to task)

TABLE OF CONTENTS

1. [Module Map](#)
2. [M1: DeterministicContext](#)
3. [M2: Primitive Types](#)
4. [M3: Compound Types — Cross-System](#)
5. [M4: Compound Types — Perception](#)
6. [M5: Compound Types — Reasoning & Decision](#)
7. [M6: Compound Types — Infrastructure](#)
8. [M7: Cross-System Traits](#)
9. [M8: Hash Helpers](#)
10. [M9: Serialization Helpers](#)
11. [M10: Time Helpers](#)
12. [M11: Math Helpers](#)
13. [M12: Linear Algebra Helpers \(nalgebra\)](#)
14. [M13: Validation Helpers](#)
15. [M14: Determinism Helpers](#)
16. [M15: Audit Helpers](#)

17. [M16: ID Helpers](#)
 18. [M17: Graph Helpers](#)
 19. [M18: Compiler & Build Configuration](#)
 20. [Aggregate Summary](#)
-

MODULE MAP

```
shared/
├─ Cargo.toml                # Dependencies: nalgebra (no-blas!), libm, sha2
├─ src/
│   ├─ lib.rs                # Re-exports
│   ├─ context.rs            # M1: DeterministicContext
│   ├─ primitives.rs         # M2: Hash256, Ed25519*, TraceId, LogicalTimest
│   └─ types/
│       ├─ cross_system.rs   # M3: ExecutionGraph, ExecutionManifest, StateF
│       ├─ perception.rs     # M4: AudioInput, ImageInput, MfccMatrix, Senso
│       ├─ reasoning.rs      # M5: StateEstimate, FiredRule, ReasoningResult
│       └─ infrastructure.rs # M6: Config, AuditRecord, StateSnapshot, Certi
│   └─ traits.rs             # M7: StateController, CryptoAttestor, Executio
│       └─ helpers/
│           ├─ hash.rs        # M8: sha256, sha256_pair, merkle_root, merkle_
│           ├─ serialize.rs   # M9: canonical_json, serialize_deterministic
│           ├─ time.rs        # M10: logical_duration, logical_time_display,
│           ├─ math.rs        # M11: clamp_f64, assert_finite, sort_stable_by
│           ├─ linalg.rs      # M12: mat_mul, mat_inverse, eigenvalues, svd,
│           ├─ validate.rs    # M13: assert_hash, validate_finite_f64, valida
│           ├─ determinism.rs # M14: verify_deterministic, byte_exact_match,
│           ├─ audit.rs       # M15: build_audit_record, validate_audit_recor
│           ├─ id.rs          # M16: trace_id_to_hex, hex_to_trace_id
│           └─ graph.rs       # M17: topological_sort, detect_cycle, validate
├─ .cargo/config.toml        # M18: FMA-disabled compiler flags
├─ clippy.toml               # M18: Banned types (HashMap, SystemTime, Uuid)
└─ ci/check_no_blas.sh      # M18: CI guard against BLAS dependency
```

M1: DeterministicContext

The single source of truth for ALL timestamps, IDs, and pipeline state. **File:**

shared/src/context.rs

Requirements

ID	Requirement	Priority	Source
REQ-M1-01	System SHALL provide DeterministicContext as the sole authority for logical time, trace IDs, and pipeline state	P0	V1/V2 resolution
REQ-M1-02	DeterministicContext.tick() SHALL advance logical_clock by exactly 1, returning new LogicalTimestamp	P0	C2.1.1
REQ-M1-03	DeterministicContext.now() SHALL return current LogicalTimestamp without mutation	P0	Pure read
REQ-M1-04	DeterministicContext.next_trace_id() SHALL produce TraceId = SHA-256(session_seed counter)[0..16]	P0	V3 resolution
REQ-M1-05	DeterministicContext.sub_id(parent, stage) SHALL produce child TraceId = SHA-256(parent stage)[0..16]	P0	Pipeline sub-IDs
REQ-M1-06	DeterministicContext SHALL NOT implement Clone (single instance per pipeline)	P0	Determinism
REQ-M1-07	DeterministicContext SHALL be !Send + !Sync (stays on pipeline thread)	P0	Thread safety
REQ-M1-08	begin_pipeline() SHALL return PipelineGuard; secure_random() SHALL panic while guard is live	P0	V3 resolution
REQ-M1-09	PipelineGuard SHALL auto-deactivate pipeline_active on Drop (RAII)	P0	Safety
REQ-M1-10	snapshot() SHALL capture (logical_clock, trace_counter, session_seed, frame_counter) for Dhruva	P0	Replay
REQ-M1-	from_replay(saved) SHALL restore context from	P0	Replay

11	ContextSnapshot identically		
REQ-M1-12	next_frame() SHALL advance frame_counter by 1 (Dristi frame-based execution)	P1	Dristi E1.5
REQ-M1-13	Same session_seed + same call sequence SHALL produce identical IDs and timestamps	P0	Core determinism

Design

Struct: `DeterministicContext` (5 fields: `logical_clock`, `trace_counter`, `session_seed`, `pipeline_active`, `frame_counter`) **Companion:** `PipelineGuard<'a>` (RAII, borrows `&mut DeterministicContext`) **Companion:** `ContextSnapshot` (Copy, for Dhruva serialization)

Construction: Only `new(seed)` or `from_replay(snapshot)`. No Default, no Clone.

Threading: `!Send + !Sync` via `PhantomData<*const ()>`. **ID derivation:** SHA-256(seed || counter) truncated to 16 bytes. **Pipeline guard:** `begin_pipeline()` asserts not nested; sets flag; `PipelineGuard.drop()` clears flag.

Public API:

```

new(session_seed: [u8;32]) → Self
from_replay(saved: &ContextSnapshot) → Self
tick(&mut self) → LogicalTimestamp
now(&self) → LogicalTimestamp
next_trace_id(&mut self) → TraceId
sub_id(&self, parent: &TraceId, stage: u16) → TraceId
next_frame(&mut self) → FrameId
current_frame(&self) → FrameId
begin_pipeline(&mut self) → PipelineGuard<'_>
is_pipeline_active(&self) → bool
snapshot(&self) → ContextSnapshot

```

Tasks

ID	Task	Hours	Maps to
TASK-M1-01	Define <code>DeterministicContext</code> struct with 5 fields, <code>!Send/!Sync</code> via <code>PhantomData</code>	2h	REQ-M1-01, M1-06, M1-07
TASK-	Implement <code>new(seed)</code> and <code>from_replay(snapshot)</code>		REQ-M1-01,

M1-02	constructors	1h	M1-11
TASK-M1-03	Implement tick() and now()	1h	REQ-M1-02, M1-03
TASK-M1-04	Implement next_trace_id() with SHA-256 derivation	2h	REQ-M1-04
TASK-M1-05	Implement sub_id(parent, stage)	1h	REQ-M1-05
TASK-M1-06	Implement next_frame() and current_frame()	0.5h	REQ-M1-12
TASK-M1-07	Implement begin_pipeline(), PipelineGuard with Drop	2h	REQ-M1-08, M1-09
TASK-M1-08	Implement snapshot() and ContextSnapshot	1h	REQ-M1-10
TASK-M1-09	Test: same seed → same ID sequence (1M iterations)	2h	REQ-M1-13
TASK-M1-10	Test: snapshot → from_replay → identical continuation	2h	REQ-M1-11
TASK-M1-11	Test: nested begin_pipeline() panics	0.5h	REQ-M1-08
TASK-M1-12	Test: !Clone, !Send, !Sync compile-fail tests	1h	REQ-M1-06, M1-07
	Module Total	16h	

M2: Primitive Types

All v1.0 primitives replaced. No f32 aliases, no String IDs, no Unix timestamps. **File:**
`shared/src/primitives.rs`

Requirements

ID	Requirement	Priority	Source
----	-------------	----------	--------

REQ-M2-01	Hash256 SHALL be [u8; 32] (SHA-256 digest)	P0	FIPS 180-4
REQ-M2-02	Ed25519Signature SHALL be [u8; 64] (correct Ed25519 size, not 256)	P0	RFC 8032
REQ-M2-03	Ed25519PublicKey SHALL be [u8; 32]	P0	RFC 8032
REQ-M2-04	Ed25519PrivateKey SHALL be [u8; 64] (expanded)	P0	RFC 8032
REQ-M2-05	TraceId SHALL be newtype [u8; 16], generated ONLY by DeterministicContext	P0	V3
REQ-M2-06	LogicalTimestamp SHALL be newtype u64, generated ONLY by DeterministicContext	P0	V1/V2
REQ-M2-07	MonotonicTimestamp SHALL be newtype u64 (nanoseconds), ONLY for Dristi RT scheduler, NEVER hashed	P1	Dristi E26
REQ-M2-08	Confidence SHALL be newtype u32 in [0, 1_000_000] representing [0.0, 1.0] fixed-point	P0	FP determinism
REQ-M2-09	Confidence SHALL provide ZERO, FULL, HIGH/MEDIUM/LOW thresholds, from_f64(), level()	P0	API
REQ-M2-10	FrameId SHALL be newtype u64, generated by DeterministicContext.next_frame()	P1	Dristi
REQ-M2-11	SpecVersion SHALL be struct {major: u16, minor: u16, patch: u16} with semver semantics	P0	Versioning
REQ-M2-	SystemId SHALL be repr(u8) enum: Praman=0, Tejas=1, Dhruva=2, Siddhanta=3, Dristi=4	P0	DGP

12			
REQ-M2-13	ArtifactType SHALL be repr(u8) enum with 6+ variants	P0	Dristi rendering
REQ-M2-14	BoundedBuffer<N> SHALL be const-generic fixed-capacity buffer, no heap after construction	P0	V5
REQ-M2-15	Standard buffer sizes SHALL be defined as constants (MAX_STATE_SIZE=16MB, MAX_SNAPSHOT_SIZE=64MB, etc.)	P0	V5
REQ-M2-16	ConfidenceLevel SHALL be enum {VeryLow, Low, Medium, High}	P0	API
REQ-M2-17	All primitive types SHALL derive Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord where applicable	P0	API
REQ-M2-18	FoundationError SHALL be comprehensive error enum covering all failure modes	P0	Error handling

Design

Cryptographic: Type aliases for [u8; N] — Hash256, Ed25519Signature, Ed25519PublicKey, Ed25519PrivateKey **Identity:** Traceld(pub [u8;16]) — Eq, Ord, Hash derives. Frameld(pub u64). **Time:** LogicalTimestamp(pub u64) — Eq, Ord. MonotonicTimestamp(pub u64) — no Ord (non-deterministic). **Confidence:** Confidence(pub u32) — constants, from_f64(), level(). Assert $\leq 1_000_000$. **Version:** SpecVersion {major, minor, patch}. FOUNDATION_VERSION = 3.0.0. **Enums:** SystemId (repr(u8), 5 variants), ArtifactType (repr(u8), 6 variants), ConfidenceLevel (4 variants). **Buffer:** BoundedBuffer<const N: usize> — data: [u8;N], len: usize. push(), extend_from_slice(), as_bytes(). **Error:** FoundationError enum — BufferOverflow, HashMismatch, SerializationError, ValidationError, ContractViolation, DimensionMismatch, NotFinite, CycleDetected, etc.

Tasks

ID	Task	Hours	Maps to

TASK-M2-01	Define cryptographic type aliases (Hash256, Ed25519*)	0.5h	REQ-M2-01 to M2-04
TASK-M2-02	Define Traceld newtype with derives + ONLY-from-context doc	1h	REQ-M2-05
TASK-M2-03	Define LogicalTimestamp, MonotonicTimestamp, Frameld newtypes	1h	REQ-M2-06, M2-07, M2-10
TASK-M2-04	Implement Confidence with constants, from_f64(), level(), assertions	2h	REQ-M2-08, M2-09, M2-16
TASK-M2-05	Define SpecVersion struct + FOUNDATION_VERSION constant	0.5h	REQ-M2-11
TASK-M2-06	Define SystemId and ArtifactType enums	0.5h	REQ-M2-12, M2-13
TASK-M2-07	Implement BoundedBuffer<N> with push(), extend_from_slice(), as_bytes()	2h	REQ-M2-14
TASK-M2-08	Define buffer size constants	0.5h	REQ-M2-15
TASK-M2-09	Define FoundationError enum with all variants	1.5h	REQ-M2-18
TASK-M2-10	Test: Confidence boundary checks, from_f64 rounding, level() thresholds	1.5h	REQ-M2-09
TASK-M2-11	Test: BoundedBuffer overflow returns error, not panic	1h	REQ-M2-14
TASK-M2-12	Test: compile-time type size assertions (Hash256=32, Ed25519Signature=64, etc.)	0.5h	REQ-M2-01 to M2-04
	Module Total	12.5h	

M3: Compound Types — Cross-System

Structs shared across all 5 DGP systems. **File:** `shared/src/types/cross_system.rs`

Requirements

ID	Requirement	Priority	Source
REQ-M3-01	ExecutionGraph SHALL contain: trace_id, graph_hash, node_count, edge_count, spec_version	P0	Tejas
REQ-M3-02	ExecutionManifest SHALL contain: trace_id, graph_fingerprint, capability_manifest, validated_at	P0	Tejas
REQ-M3-03	StateFingerprint SHALL contain: trace_id, state_hash, model_weight_hash, environment_hash, captured_at	P0	Dhruva
REQ-M3-04	ExecutionCertificate SHALL contain: trace_id, audit_hash, signature, signer, signed_at, spec_version	P0	Siddhanta
REQ-M3-05	LedgerEntry SHALL contain: sequence, prev_hash, entry_hash, certificate	P0	Siddhanta
REQ-M3-06	DecisionOutput SHALL contain: trace_id, decision_hash, confidence, proof_hash	P0	Praman
REQ-M3-07	RenderRequest SHALL contain: trace_id, certificate, artifact_type, logical_time	P0	Dristi
REQ-M3-08	RenderedArtifact SHALL contain: trace_id, artifact_hash, artifact_type, rendered_at	P0	Dristi
REQ-M3-09	ALL structs SHALL use LogicalTimestamp (not Timestamp), TraceId (not String), Confidence (not f32)	P0	v3.0
REQ-M3-10	ALL collections in structs SHALL be BTreeMap (not HashMap)	P0	W4

Design

8 structs with deterministic fields. No HashMap, no String IDs, no f32, no wall-clock time.
Every struct: Debug + Clone. StateFingerprint: also Copy.

Tasks

ID	Task	Hours	Maps to
TASK-M3-01	Define ExecutionGraph struct	0.5h	REQ-M3-01
TASK-M3-02	Define ExecutionManifest struct	0.5h	REQ-M3-02
TASK-M3-03	Define StateFingerprint struct (Copy)	0.5h	REQ-M3-03
TASK-M3-04	Define ExecutionCertificate struct	0.5h	REQ-M3-04
TASK-M3-05	Define LedgerEntry struct	0.5h	REQ-M3-05
TASK-M3-06	Define DecisionOutput struct	0.5h	REQ-M3-06
TASK-M3-07	Define RenderRequest and RenderedArtifact structs	0.5h	REQ-M3-07, M3-08
TASK-M3-08	Test: all structs use correct types, no HashMap anywhere	1h	REQ-M3-09, M3-10
	Module Total	4.5h	

M4: Compound Types — Perception

Types for Praman's 7 perception modalities. **File:** `shared/src/types/perception.rs`

Requirements

--	--	--	--

ID	Requirement	Priority	Source
REQ-M4-01	AudioInput SHALL contain: samples (Vec<f64>), sample_rate (u32), duration_ms (u32)	P0	Praman E2
REQ-M4-02	MfccMatrix SHALL use nalgebra::DMatrix<f64> (not Vec<Vec<f64>>)	P0	Julia elimination
REQ-M4-03	ImageInput SHALL contain: pixels (Vec<f64>), width/height/channels (u32)	P0	Praman E3
REQ-M4-04	SensorReading SHALL use u32 sensor_id, f64 value, LogicalTimestamp, Confidence, u16 unit_code	P0	v3.0
REQ-M4-05	PerceptionResult SHALL use f64 features (not f32), Confidence (not f32), LogicalTimestamp	P0	v3.0
REQ-M4-06	ALL perception types SHALL use f64 (not f32) for numerical values	P0	v3.0
REQ-M4-07	Modality SHALL be an enum (Audio, Vision, Text, Sensor, Geospatial, Temporal, MultiModal)	P0	Praman E2-E4

Design

6 structs + 1 enum. All f64, nalgebra::DMatrix where matrix is needed. No f32, no String IDs.

Tasks

ID	Task	Hours	Maps to
TASK-M4-01	Define AudioInput, MfccMatrix (DMatrix<f64>)	1h	REQ-M4-01, M4-02
TASK-M4-02	Define ImageInput	0.5h	REQ-M4-03

TASK-M4-03	Define SensorReading with u32/f64/LogicalTimestamp/Confidence	0.5h	REQ-M4-04
TASK-M4-04	Define PerceptionResult, Modality enum	1h	REQ-M4-05, M4-07
TASK-M4-05	Test: no f32 anywhere in perception types (grep test)	0.5h	REQ-M4-06
	Module Total	3.5h	

M5: Compound Types — Reasoning & Decision

Types for Praman reasoning engine, rules, forecasting, decision output. **File:**
`shared/src/types/reasoning.rs`

Requirements

ID	Requirement	Priority	Source
REQ-M5-01	StateEstimate SHALL use nalgebra::DVector<f64> for state and DMatrix<f64> for covariance	P0	Julia elim
REQ-M5-02	FiredRule SHALL use u32 rule_id (not String), u32 priority, Vec<u32> conditions, Confidence	P0	v3.0
REQ-M5-03	RuleAction SHALL use u32 fact_id, ActionValue enum (Int, Float, Bool, Text)	P0	v3.0
REQ-M5-04	TemporalForecast SHALL contain predictions, confidence_lower/upper as Vec<f64>, horizon_steps	P0	Praman E6
REQ-M5-05	ReasoningResult SHALL aggregate StateEstimate + FiredRules + TemporalForecast + Hash256	P0	Praman E5-E7
REQ-M5-	DpsInput SHALL contain optional AudioInput/ImageInput/text/sensors + LogicalTimestamp	P0	Praman top-

06			level
REQ-M5-07	DpsOutput SHALL use Traceld (not DecisionId=String), ExecutionCertificate (not Signature256)	P0	v3.0
REQ-M5-08	ActionItem SHALL use u32 action_code, BTreeMap<u32, ActionValue> parameters	P0	v3.0

Design

8 structs + 2 enums. nalgebra::DVector/DMatrix for state estimation. BTreeMap for parameters. u32 IDs throughout.

Tasks

ID	Task	Hours	Maps to
TASK-M5-01	Define StateEstimate (DVector + DMatrix + Confidence)	1h	REQ-M5-01
TASK-M5-02	Define FiredRule, RuleAction, ActionValue enum	1h	REQ-M5-02, M5-03
TASK-M5-03	Define TemporalForecast	0.5h	REQ-M5-04
TASK-M5-04	Define ReasoningResult	0.5h	REQ-M5-05
TASK-M5-05	Define DpsInput, DpsOutput with correct v3.0 types	1h	REQ-M5-06, M5-07
TASK-M5-06	Define ActionItem with BTreeMap	0.5h	REQ-M5-08
TASK-M5-07	Test: no HashMap, no String IDs, no f32 in any type	1h	All
	Module Total	5.5h	

M6: Compound Types — Infrastructure

Config, AuditRecord, StateSnapshot, Certificate, GraphNode, CompilationResult. **File:**
`shared/src/types/infrastructure.rs`

Requirements

ID	Requirement	Priority	Source
REQ-M6-01	Config SHALL use BTreeMap<String, ConfigValue> (not HashMap), SpecVersion, Hash256, DomainTag	P0	v3.0
REQ-M6-02	AuditRecord SHALL use Traceld (not DecisionId), LogicalTimestamp, ExecutionCertificate	P0	v3.0
REQ-M6-03	StateSnapshot SHALL use BTreeMap<u32, Hash256> model_hashes, SystemId, LogicalTimestamp	P0	Dhruva
REQ-M6-04	Certificate SHALL use Ed25519Signature (64 bytes), LogicalTimestamp, optional chain_link	P0	Siddhanta
REQ-M6-05	GraphNode SHALL use u32 id/module_id, u64 budget_us/memory_bytes	P0	Tejas
REQ-M6-06	CompilationResult SHALL use BTreeMap<u32, RwSet>	P0	Tejas
REQ-M6-07	DomainTag SHALL be an enum (not String)	P0	v3.0

Design

6 structs + 2 enums + ConfigValue enum. All BTreeMap, u32 IDs, LogicalTimestamp.

Tasks

ID	Task	Hours	Maps to
TASK-M6-01	Define Config, DomainTag enum, DgpFlag, ConfigValue	1h	REQ-M6-01, M6-07
TASK-M6-02	Define AuditRecord with v3.0 types	0.5h	REQ-M6-02
TASK-M6-03	Define StateSnapshot with BTreeMap	0.5h	REQ-M6-03
TASK-M6-04	Define Certificate with Ed25519Signature	0.5h	REQ-M6-04
TASK-M6-05	Define GraphNode, CompilationResult, RwSet, ScheduleEntry	1h	REQ-M6-05, M6-06
TASK-M6-06	Test: zero HashMap in entire types/ directory	0.5h	All
	Module Total	4h	

M7: Cross-System Traits

5 traits defining inter-system contracts. All receive &DeterministicContext. **File:**

`shared/src/traits.rs`

Requirements

ID	Requirement	Priority	Source
REQ-M7-01	StateController trait SHALL define capture_state(&ctx) → StateFingerprint, verify_replay(&fp) → bool	P0	Dhruva
REQ-M7-02	CryptoAttestor trait SHALL define sign(hash, &ctx) → ExecutionCertificate, verify(&cert) → bool, append_ledger(cert) → LedgerEntry	P0	Siddhanta
REQ-M7-03	ExecutionValidator trait SHALL define validate_graph(&graph, &ctx) → ExecutionManifest	P0	Tejas

REQ-M7-04	ArtifactRenderer trait SHALL define render(&request) → RenderedArtifact	P0	Dristi
REQ-M7-05	DecisionEngine trait SHALL define process(&input, &mut ctx) → DecisionOutput	P0	Praman
REQ-M7-06	ALL traits SHALL be Send + Sync (object-safe for Transport)	P0	DGP Transport
REQ-M7-07	Praman's DecisionEngine SHALL receive &mut DeterministicContext (calls tick, next_trace_id, begin_pipeline)	P0	Praman is the driver
REQ-M7-08	ALL other traits SHALL receive &DeterministicContext (read-only)	P0	Receivers

Design

5 trait definitions. All `Send + Sync`. `DecisionEngine` gets `&mut ctx` (it drives the pipeline). Others get `&ctx` or use data already containing `LogicalTimestamp`.

Tasks

ID	Task	Hours	Maps to
TASK-M7-01	Define StateController trait	0.5h	REQ-M7-01
TASK-M7-02	Define CryptoAttestor trait	1h	REQ-M7-02
TASK-M7-03	Define ExecutionValidator trait	0.5h	REQ-M7-03
TASK-M7-04	Define ArtifactRenderer trait	0.5h	REQ-M7-04
TASK-M7-05	Define DecisionEngine trait with &mut ctx	0.5h	REQ-M7-05, M7-07

TASK-M7-06	Test: all traits are object-safe (dyn Trait compiles)	1h	REQ-M7-06
TASK-M7-07	Test: trait method signatures compile with correct context mutability	0.5h	REQ-M7-07, M7-08
	Module Total	4.5h	

M8: Hash Helpers

SHA-256 hashing, Merkle trees. Used by ALL 5 systems. **File:**

`shared/src/helpers/hash.rs`

Requirements

ID	Requirement	Priority	Source
REQ-M8-01	hash_sha256(data) SHALL return FIPS 180-4 compliant SHA-256 digest	P0	Siddhanta E1.3
REQ-M8-02	hash_sha256_concat(parts) SHALL hash concatenation of multiple slices	P0	Chain hashing
REQ-M8-03	hash_eq(a, b) SHALL compare in constant time (timing-attack resistant)	P0	Siddhanta E6.4
REQ-M8-04	hash_to_hex(h) / hex_to_hash(hex) SHALL convert bidirectionally	P0	Display
REQ-M8-05	merkle_root(hashes) SHALL compute deterministic Merkle tree root	P0	Siddhanta E1.8
REQ-M8-06	merkle_verify(leaf, proof, root) SHALL verify inclusion path	P0	Siddhanta E1.8
REQ-M8-07	ALL hash functions SHALL produce identical output across x86, ARM64, RISC-V, WASM	P0	Platform

Design

Uses `sha2` crate. Constant-time comparison via `subtle` crate or manual constant-time

loop. Merkle tree: ordered leaves, SHA-256 pair function, $O(\log n)$ proof.

Tasks

ID	Task	Hours	Maps to
TASK-M8-01	Implement <code>hash_sha256()</code> using <code>sha2</code> crate	1h	REQ-M8-01
TASK-M8-02	Implement <code>hash_sha256_concat()</code>	0.5h	REQ-M8-02
TASK-M8-03	Implement <code>hash_eq()</code> with constant-time comparison	1h	REQ-M8-03
TASK-M8-04	Implement <code>hash_to_hex()</code> , <code>hex_to_hash()</code>	0.5h	REQ-M8-04
TASK-M8-05	Implement <code>merkle_root()</code>	2h	REQ-M8-05
TASK-M8-06	Implement <code>merkle_verify()</code>	1.5h	REQ-M8-06
TASK-M8-07	Test: FIPS 180-4 test vectors pass	1h	REQ-M8-01
TASK-M8-08	Test: <code>merkle_root</code> deterministic (same leaves → same root)	1h	REQ-M8-05
TASK-M8-09	Test: merkle round-trip (build proof → verify → true)	1h	REQ-M8-06
TASK-M8-10	Test: constant-time comparison timing analysis	1h	REQ-M8-03
	Module Total	10.5h	

M9: Serialization Helpers

Canonical JSON, deterministic binary serialization. **File:**

```
shared/src/helpers/serialize.rs
```

Requirements

ID	Requirement	Priority	Source
REQ-M9-01	canonical_json(value) SHALL produce sorted-key, no-whitespace JSON	P0	Determinism
REQ-M9-02	canonical_json_bounded(value, buf) SHALL write to BoundedBuffer (no heap)	P0	V5
REQ-M9-03	serialize_deterministic(value) SHALL produce big-endian, declaration-order binary	P0	Wire format
REQ-M9-04	deserialize_deterministic(bytes) SHALL round-trip: deserialize(serialize(x)) == x	P0	Correctness
REQ-M9-05	DeterministicSerialize / DeterministicDeserialize traits SHALL be defined for derivation	P0	Trait
REQ-M9-06	msgpack helpers SHALL NOT exist (Julia eliminated)	P0	v3.0

Design

Uses `serde_json` with sorted keys for canonical JSON. Custom binary serializer: big-endian, field declaration order, no padding, no alignment. Derive macro or manual impl for `DeterministicSerialize`.

Tasks

ID	Task	Hours	Maps to
TASK-M9-01	Implement <code>canonical_json()</code> with sorted keys	1.5h	REQ-M9-01
TASK-M9-02	Implement <code>canonical_json_bounded()</code>	1.5h	REQ-M9-02
TASK-M9-03	Define <code>DeterministicSerialize</code> / <code>DeterministicDeserialize</code> traits	1h	REQ-M9-05
TASK-M9-04	Implement <code>serialize_deterministic()</code>	2h	REQ-M9-03

TASK-M9-05	Implement <code>deserialize_deterministic()</code>	2h	REQ-M9-04
TASK-M9-06	Test: <code>canonical_json</code> produces sorted keys, no whitespace	1h	REQ-M9-01
TASK-M9-07	Test: round-trip for all compound types	2h	REQ-M9-04
TASK-M9-08	Test: bounded buffer overflow returns error	0.5h	REQ-M9-02
TASK-M9-09	Verify: no msgpack functions exist	0.5h	REQ-M9-06
	Module Total	12h	

M10: Time Helpers

Logical time only. NO wall-clock functions. **File:** `shared/src/helpers/time.rs`

Requirements

ID	Requirement	Priority	Source
REQ-M10-01	<code>logical_duration(start, end)</code> SHALL return <code>end - start</code> with <code>saturating_sub</code>	P0	All systems
REQ-M10-02	<code>logical_time_display(ts)</code> SHALL return "LT:0000000042" format (10-digit zero-padded)	P0	Logging
REQ-M10-03	<code>logical_time_cmp(a, b)</code> SHALL return Ordering	P0	Sorting
REQ-M10-04	<code>timestamp_now()</code> SHALL NOT EXIST	P0	V1/V2
REQ-M10-05	<code>timestamp_to_iso()</code> / <code>iso_to_timestamp()</code> SHALL NOT EXIST	P0	v3.0

Design

3 functions. All pure. No system clock calls. No `std::time` imports.

Tasks

ID	Task	Hours	Maps to
TASK-M10-01	Implement <code>logical_duration()</code>	0.5h	REQ-M10-01
TASK-M10-02	Implement <code>logical_time_display()</code>	0.5h	REQ-M10-02
TASK-M10-03	Implement <code>logical_time_cmp()</code>	0.5h	REQ-M10-03
TASK-M10-04	Test: duration saturates at 0 (no underflow)	0.5h	REQ-M10-01
TASK-M10-05	Verify: no <code>timestamp_now</code> , no <code>SystemTime</code> import, no wall-clock	0.5h	REQ-M10-04, M10-05
	Module Total	2.5h	

M11: Math Helpers

Pure Rust math. FMA disabled. No Julia. No f32 in core types. **File:**

`shared/src/helpers/math.rs`

Requirements

ID	Requirement	Priority	Source
REQ-M11-01	<code>clamp_f64(value, min, max)</code> SHALL clamp to <code>[min, max]</code>	P0	General
REQ-M11-02	<code>assert_finite(value, context)</code> SHALL panic if NaN or Inf	P0	Boundary
REQ-			

M11-03	assert_all_finite(values, context) SHALL validate entire slice	P0	Boundary
REQ-M11-04	confidence_level(c) SHALL return ConfidenceLevel per thresholds	P0	Praman
REQ-M11-05	sort_stable_by_key() SHALL guarantee deterministic order	P0	V6
REQ-M11-06	f64_total_cmp(a, b) SHALL provide total ordering (NaN sorts to end, -0.0 == +0.0)	P0	Sorting
REQ-M11-07	deterministic_parallel_map(items, f) SHALL collect results in INPUT ORDER (not completion order)	P0	V6
REQ-M11-08	ieee754_canonical() SHALL NOT EXIST (unnecessary with pure Rust)	P0	v3.0

Design

7 functions. `deterministic_parallel_map` uses rayon with indexed parallel iterator and pre-allocated output vec. `f64_total_cmp` uses `f64::total_cmp()` (stable since Rust 1.62).

Tasks

ID	Task	Hours	Maps to
TASK-M11-01	Implement <code>clamp_f64()</code>	0.5h	REQ-M11-01
TASK-M11-02	Implement <code>assert_finite()</code> , <code>assert_all_finite()</code>	1h	REQ-M11-02, M11-03
TASK-M11-03	Implement <code>confidence_level()</code>	0.5h	REQ-M11-04
TASK-M11-04	Implement <code>sort_stable_by_key()</code>	0.5h	REQ-M11-05

TASK-M11-05	Implement f64_total_cmp()	0.5h	REQ-M11-06
TASK-M11-06	Implement deterministic_parallel_map() with positional collection	3h	REQ-M11-07
TASK-M11-07	Test: parallel_map output order == input order (1M items)	1.5h	REQ-M11-07
TASK-M11-08	Test: NaN/Inf triggers panic in assert_finite	0.5h	REQ-M11-02
TASK-M11-09	Verify: no ieee754_canonical function exists	0.5h	REQ-M11-08
	Module Total	8.5h	

M12: Linear Algebra Helpers

Pure Rust nalgebra wrappers + libm transcendentals. No BLAS. Bit-exact. **File:**

`shared/src/helpers/linalg.rs`

Requirements

ID	Requirement	Priority	Source
REQ-M12-01	mat_mul(a, b) SHALL compute matrix multiply using nalgebra (no BLAS)	P0	Praman E5-E7
REQ-M12-02	mat_inverse(a) SHALL compute inverse via LU decomposition (pure Rust)	P0	Kalman filter
REQ-M12-03	eigenvalues(a) SHALL compute eigenvalues via QR algorithm (pure Rust)	P0	SSM
REQ-M12-04	solve_linear(a, b) SHALL solve $Ax=b$ via LU (pure Rust)	P0	Reasoning
REQ-M12-05	cholesky(a) SHALL compute Cholesky decomposition for PSD matrices	P0	Kalman
REQ-	is_positive_semidefinite(a) SHALL validate PSD	P0	Validation

M12-06	property		
REQ-M12-07	svd(a) SHALL compute deterministic SVD	P0	Learning
REQ-M12-08	det_sin/cos/exp/log/sqrt SHALL use libm (pure Rust, bit-exact cross-platform)	P0	Physics
REQ-M12-09	nalgebra SHALL be compiled WITHOUT blas feature (CI-enforced)	P0	Determinism
REQ-M12-10	ALL linear algebra operations SHALL be bit-exact across x86/ARM64/RISC-V	P0	Determinism

Design

7 matrix functions wrapping nalgebra. 5 transcendental functions wrapping libm. All pure Rust. No BLAS, no LAPACK, no MKL. Bit-exact by construction.

Tasks

ID	Task	Hours	Maps to
TASK-M12-01	Implement mat_mul()	1h	REQ-M12-01
TASK-M12-02	Implement mat_inverse() via nalgebra LU	1.5h	REQ-M12-02
TASK-M12-03	Implement eigenvalues() via nalgebra QR	1.5h	REQ-M12-03
TASK-M12-04	Implement solve_linear()	1h	REQ-M12-04
TASK-M12-05	Implement cholesky()	1h	REQ-M12-05
TASK-M12-06	Implement is_positive_semidefinite()	1h	REQ-M12-06
TASK-M12-07	Implement svd()	1.5h	REQ-M12-07

TASK-M12-08	Implement det_sin/cos/exp/log/sqrt wrapping libm	1h	REQ-M12-08
TASK-M12-09	Test: mat_mul bit-exact across runs (N-run harness)	2h	REQ-M12-10
TASK-M12-10	Test: inverse(A) * A $\approx$ I within deterministic tolerance	1h	REQ-M12-02
TASK-M12-11	Test: cholesky fails for non-PSD input	0.5h	REQ-M12-05
TASK-M12-12	Test: libm transcendentals match known values	1h	REQ-M12-08
	Module Total	14h	

M13: Validation Helpers

Hash assertions, finite checks, dimension checks, spec version checks. **File:**

`shared/src/helpers/validate.rs`

Requirements

ID	Requirement	Priority	Source
REQ-M13-01	assert_hash(data, expected) SHALL recompute SHA-256 and compare	P0	Integrity
REQ-M13-02	validate_finite_f64(values, context) SHALL return Error if any NaN/Inf	P0	Boundary
REQ-M13-03	validate_psd(matrix) SHALL check positive semi-definiteness	P0	Kalman
REQ-M13-04	validate_dimensions(actual, expected, name) SHALL return Error on mismatch	P0	Shape safety
REQ-M13-05	validate_spec_version(actual, expected) SHALL check major version compatibility	P0	Versioning

Tasks

ID	Task	Hours	Maps to
TASK-M13-01	Implement assert_hash()	1h	REQ-M13-01
TASK-M13-02	Implement validate_finite_f64()	0.5h	REQ-M13-02
TASK-M13-03	Implement validate_psd()	1h	REQ-M13-03
TASK-M13-04	Implement validate_dimensions()	0.5h	REQ-M13-04
TASK-M13-05	Implement validate_spec_version()	0.5h	REQ-M13-05
TASK-M13-06	Test: hash mismatch returns error, not panic	0.5h	REQ-M13-01
TASK-M13-07	Test: NaN/Inf correctly detected in slices	0.5h	REQ-M13-02
	Module Total	4.5h	

M14: Determinism Helpers

N-run verification harness, byte-exact comparison, output hashing. **File:**
`shared/src/helpers/determinism.rs`

Requirements

ID	Requirement	Priority	Source
REQ-M14-01	verify_deterministic(f, runs) SHALL execute f N times and verify all output hashes match	P0	Core
REQ-M14-02	byte_exact_match(a, b) SHALL compare two byte slices	P0	Testing
REQ-M14-03	output_hash(output) SHALL hash any DeterministicSerialize value	P0	Hashing

Tasks

--	--	--	--

ID	Task	Hours	Maps to
TASK-M14-01	Implement verify_deterministic()	2h	REQ-M14-01
TASK-M14-02	Implement byte_exact_match()	0.5h	REQ-M14-02
TASK-M14-03	Implement output_hash()	0.5h	REQ-M14-03
TASK-M14-04	Test: verify_deterministic detects non-determinism (inject randomness)	1h	REQ-M14-01
	Module Total	4h	

M15: Audit Helpers

Build and validate audit records. **File:** `shared/src/helpers/audit.rs`

Requirements

ID	Requirement	Priority	Source
REQ-M15-01	build_audit_record() SHALL construct AuditRecord from component hashes	P0	Governance
REQ-M15-02	validate_audit_record() SHALL verify all hashes present and certificate valid	P0	Governance
REQ-M15-03	ALL fields SHALL use v3.0 types (Traceld, LogicalTimestamp, ExecutionCertificate)	P0	v3.0

Tasks

ID	Task	Hours	Maps to
TASK-M15-01	Implement build_audit_record()	1h	REQ-M15-01

TASK-M15-02	Implement <code>validate_audit_record()</code>	1h	REQ-M15-02
TASK-M15-03	Test: round-trip build → validate succeeds	0.5h	REQ-M15-01, M15-02
TASK-M15-04	Test: tampered record fails validation	0.5h	REQ-M15-02
	Module Total	3h	

M16: ID Helpers

Display/parse only. All ID generation via `DeterministicContext`. **File:**

`shared/src/helpers/id.rs`

Requirements

ID	Requirement	Priority	Source
REQ-M16-01	<code>trace_id_to_hex(id)</code> SHALL format <code>TraceId</code> as 32-char hex string	P0	Display
REQ-M16-02	<code>hex_to_trace_id(hex)</code> SHALL parse 32-char hex to <code>TraceId</code>	P0	Parsing
REQ-M16-03	<code>generate_decision_id()</code> SHALL NOT EXIST	P0	v3.0
REQ-M16-04	<code>generate_snapshot_id()</code> SHALL NOT EXIST	P0	v3.0

Tasks

ID	Task	Hours	Maps to
TASK-M16-01	Implement <code>trace_id_to_hex()</code>	0.5h	REQ-M16-01
TASK-M16-02	Implement <code>hex_to_trace_id()</code>	0.5h	REQ-M16-02

TASK-M16-03	Test: round-trip hex conversion	0.5h	All
TASK-M16-04	Verify: no generate_*_id() functions exist	0.5h	REQ-M16-03, M16-04
	Module Total	2h	

M17: Graph Helpers

DAG operations for Tejas execution graphs. **File:** `shared/src/helpers/graph.rs`

Requirements

ID	Requirement	Priority	Source
REQ-M17-01	topological_sort(nodes, edges) SHALL return valid topological ordering	P0	Tejas
REQ-M17-02	detect_cycle(nodes, edges) SHALL return cycle if present	P0	Tejas
REQ-M17-03	validate_dag(nodes, edges) SHALL return Ok if valid DAG, Err if cyclic	P0	Tejas
REQ-M17-04	topological_sort SHALL be deterministic (same graph → same order)	P0	Determinism

Tasks

ID	Task	Hours	Maps to
TASK-M17-01	Implement topological_sort() (Kahn's algorithm with BTreeSet for determinism)	2h	REQ-M17-01, M17-04
TASK-M17-02	Implement detect_cycle()	1.5h	REQ-M17-02
TASK-M17-03	Implement validate_dag()	0.5h	REQ-M17-03

TASK-M17-04	Test: known DAG → expected topological order	1h	REQ-M17-01
TASK-M17-05	Test: cyclic graph detected	0.5h	REQ-M17-02
TASK-M17-06	Test: deterministic order across 1K runs	1h	REQ-M17-04
	Module Total	6.5h	

M18: Compiler & Build Configuration

FMA flags, clippy bans, CI guards, Cargo.toml dependencies. **Files:**

`.cargo/config.toml` , `clippy.toml` , `ci/check_no_blas.sh` , `Cargo.toml`

Requirements

ID	Requirement	Priority	Source
REQ-M18-01	Rust compiler SHALL disable FMA globally: <code>--fp-contract=off</code>	P0	FP determinism
REQ-M18-02	x86_64 target SHALL use <code>target-cpu=x86-64</code> (no AVX FMA)	P0	Platform
REQ-M18-03	aarch64 target SHALL use <code>target-cpu=generic</code> (no NEON FMA)	P0	Platform
REQ-M18-04	clippy SHALL ban HashMap, HashSet, SystemTime, Instant (for deterministic paths), Uuid	P0	W4
REQ-M18-05	CI SHALL check that nalgebra has NO blas/lapack/openblas/mkl feature enabled	P0	Determinism
REQ-M18-	Cargo.toml SHALL depend on nalgebra (no-default-	P0	Dependencies

06	features), libm, sha2, ed25519-dalek		
REQ-M18-07	nalgebra SHALL be compiled WITHOUT blas feature explicitly	P0	Critical

Design

`.cargo/config.toml` : Global rustflags disabling FMA for all targets. `clippy.toml` : `disallowed-types` for HashMap, HashSet, SystemTime, Instant, Uuid with reasons.
`ci/check_no_blas.sh` : `cargo tree -f '{f}' | grep blas|lapack|openblas|mkl → fail CI` if found. `Cargo.toml` : `nalgebra = { version = "0.33", default-features = false }, libm = "0.2" .`

Tasks

ID	Task	Hours	Maps to
TASK-M18-01	Create <code>.cargo/config.toml</code> with FMA flags for x86/arm/generic	1h	REQ-M18-01, M18-02, M18-03
TASK-M18-02	Create <code>clippy.toml</code> with banned types	0.5h	REQ-M18-04
TASK-M18-03	Create <code>ci/check_no_blas.sh</code> script	0.5h	REQ-M18-05
TASK-M18-04	Configure Cargo.toml with correct dependencies (nalgebra no-default-features)	0.5h	REQ-M18-06, M18-07
TASK-M18-05	Test: <code>cargo clippy</code> catches HashMap usage	0.5h	REQ-M18-04
TASK-M18-06	Test: CI script detects blas if injected as test	0.5h	REQ-M18-05
TASK-M18-07	Test: FMA disabled verified ($a*b+c \neq \text{fma}(a,b,c)$ for known edge case)	1h	REQ-M18-01
	Module Total	4.5h	

AGGREGATE SUMMARY

Totals by Module

Module	Requirements	Tasks	Hours	Priority
M1: DeterministicContext	13	12	16h	P0 — build FIRST
M2: Primitive Types	18	12	12.5h	P0 — build FIRST
M3: Compound Types (Cross-System)	10	8	4.5h	P0 — build FIRST
M4: Compound Types (Perception)	7	5	3.5h	P0
M5: Compound Types (Reasoning)	8	7	5.5h	P0
M6: Compound Types (Infrastructure)	7	6	4h	P0
M7: Cross-System Traits	8	7	4.5h	P0 — build FIRST
M8: Hash Helpers	7	10	10.5h	P0 — build FIRST
M9: Serialization Helpers	6	9	12h	P0
M10: Time Helpers	5	5	2.5h	P0
M11: Math Helpers	8	9	8.5h	P0
M12: Linear Algebra (nalgebra)	10	12	14h	P0
M13: Validation Helpers	5	7	4.5h	P0
M14: Determinism Helpers	3	4	4h	P0
M15: Audit Helpers	3	4	3h	P1
M16: ID Helpers	4	4	2h	P1
M17: Graph Helpers	4	6	6.5h	P1

M18: Compiler & Build Config	7	7	4.5h	P0 — build FIRST
TOTAL	133	134	122.5h	

Build Order (Critical Path)

Sprint 0 (Week 1): Foundation Setup — 37.5h

- └─ M18: Compiler & Build Config (4.5h) ← FIRST: all code depends on FMA fl
- └─ M2: Primitive Types (12.5h) ← SECOND: everything uses these
- └─ M1: DeterministicContext (16h) ← THIRD: time/ID authority
- └─ M7: Cross-System Traits (4.5h) ← FOURTH: system interfaces

Sprint 1 (Week 2): Core Helpers — 33h

- └─ M8: Hash Helpers (10.5h) ← Used by ALL systems
- └─ M9: Serialization Helpers (12h) ← Used by ALL systems
- └─ M10: Time Helpers (2.5h)
- └─ M11: Math Helpers (8.5h)

Sprint 2 (Week 3): Types + Advanced Helpers — 31.5h

- └─ M3: Compound Types Cross-System (4.5h)
- └─ M4: Compound Types Perception (3.5h)
- └─ M5: Compound Types Reasoning (5.5h)
- └─ M6: Compound Types Infrastructure (4h)
- └─ M12: Linear Algebra (14h)

Sprint 3 (Week 4): Validation + Remaining — 20.5h

- └─ M13: Validation Helpers (4.5h)
- └─ M14: Determinism Helpers (4h)
- └─ M15: Audit Helpers (3h)
- └─ M16: ID Helpers (2h)
- └─ M17: Graph Helpers (6.5h)

Total: 122.5 hours ÷ 8h/day = ~15.3 dev-days ≈ 3 weeks for 1 developer

Dependency Map

```

M18 (Compiler Config)
  ↓
M2 (Primitives)
  ↓
M1 (DeterministicContext) — depends on M2 (LogicalTimestamp, TraceId, FrameId)
  ↓

```

M7 (Traits) ————— depends on M2 + M3 (all cross-system types)

↓

M3 (Compound Types: Cross) — depends on M2

M4 (Compound Types: Percep) — depends on M2 + M12 (nalgebra)

M5 (Compound Types: Reason) — depends on M2 + M12 (nalgebra)

M6 (Compound Types: Infra) — depends on M2 + M3

↓

M8 (Hash) — depends on M2 (Hash256)

M9 (Serialize) — depends on M2 (BoundedBuffer, all types)

M10 (Time) — depends on M2 (LogicalTimestamp)

M11 (Math) — depends on M2 (Confidence)

M12 (LinAlg) — depends on nalgebra crate

M13 (Validate) — depends on M2, M8, M12

M14 (Determinism) — depends on M8, M9

M15 (Audit) — depends on M2, M3

M16 (ID) — depends on M2 (TraceId)

M17 (Graph) — depends on M2 (FoundationError)

Deleted Types & Helpers (v3.0 Cleanup Verification)

Deleted Item	Replacement	Verify Task
Timestamp = i64	LogicalTimestamp	grep test
Confidence = f32	Confidence(u32)	grep test
Signature256 = [u8; 256]	Ed25519Signature = [u8; 64]	grep test
DecisionId = String	TraceId = [u8; 16]	grep test
ModuleId = String	u32	grep test
DomainTag = String	DomainTag enum	grep test
timestamp_now()	DeterministicContext.now()	grep test
duration_ms()	logical_duration()	grep test
ieee754_canonical()	DELETED (unnecessary)	grep test
to_msgpack() / from_msgpack()	DELETED (Julia eliminated)	grep test
generate_decision_id()	DeterministicContext.next_trace_id()	grep test

<code>HashMap<K, V></code> (in structs)	<code>BTreeMap<K, V></code>	clippy ban
<code>Vec<Vec<f64>></code> (matrices)	<code>nalgebra::DMatrix<f64></code>	code review
<code>f32</code> (in core types)	<code>f64</code> everywhere	grep test
All <code>julia_types.rs</code> (18 types)	DELETED	file absence check

Foundation & Utils v3.0: 18 modules, 133 requirements, 134 tasks, 122.5 hours. Zero HashMap. Zero Julia. Zero wall-clock. One DeterministicContext rules them all.